ResumeForge ATS — AI Resume Generator & JD Matcher

Comprehensive end-to-end record of developer prompts, architectural decisions, core algorithms, frontend development, automated test results, GitHub repository integration, and live cloud deployment.

Project:	ResumeForge ATS	**Date:**	September 4, 2026
GitHub:	github.com/mupanpruthvi/ATS-Resume	**Developer:**	mupanpruthvi
Live URL:	**https://resume-forge-ats.onrender.com**	**Environment :**	Python 3.14 / FastAPI / Render

Project Execution Summary

- **Turn 1 (Initial Requirement):** User requested an ATS resume builder where students can copy JD text or JD links from anywhere, upload an existing CV (or type skills/details), and generate a tailored ATS resume with an ATS match score.
- **Phase 1 (Architecture & Approval):** Verified Python 3.14 environment, installed dependencies (FastAPI, uvicorn, beautifulsoup4, pypdf, python-docx, requests), and obtained user review and approval on the implementation plan.
- **Phase 2 (Engine & Parsers):** Implemented parsers.py (document extraction & web scraper) and ats_engine.py (scoring algorithm, canonical skill taxonomy, bullet point enhancer, and Word .docx generator).
- **Phase 3 (Frontend & Server):** Built responsive 3-step wizard with radial ATS score gauge, Before vs. After comparison, keyword inspector with 1-click '+' skill injection, live in-place resume editor, and print/PDF stylesheet.
- **Phase 4 (Automated Testing):** Authored test_ats.py and test_api.py; verified document parsing, score calculation, and all API endpoints (100% pass).
- **Phase 5 (GitHub Publishing):** Installed Git and GitHub CLI, initialized repository, authored requirements.txt, and published branch main to **github.com/mupanpruthvi/ATS-Resume**.
- **Phase 6 (Cloud Deployment):** Created render.yaml blueprint and Procfile, enabled 1-click cloud deployment, and updated GitHub repo homepage with the live link **https://resume-forge-ats.onrender.com**.

Detailed Chronological Conversation Log

TURN #1 — 2026-09-04 10:17:54

USER PROMPT:
Create a website where students can copy jd or jd links from any where and after that students can upload their old Cv or they can tye the skills or what ever the details required for CV now after doing this the website should generate the ATS resume according to jD and should even show the ats score after preparing

AI ASSISTANT RESPONSE:
I have started installing the required Python dependencies (**fastapi**, **uvicorn**, **beautifulsoup4**, **pypdf**, **python-docx**, **requests**) in the background to support PDF/DOCX parsing, web scraping for job description links, ATS keyword scoring,

and resume generation. I will prepare the complete implementation plan as soon as this completes.

I have prepared the implementation plan for the **ATS Resume Generator & Job Description Matcher** web application.

Please review the plan in [**implementation_plan.md**](file:///C:/Users/EdunetFoundation/.gemini/antigravity/brain/a1c97eaa-eed3-449d-bc27-5a68bc0989e3/implementation_plan.md) and let me know if you would like me to proceed with building the application or if you have any adjustments!

Key Operations: *■ list_dir | ■ run_command | ■ manage_task | ■ write_to_file*

TURN #2 — 2026-09-04 10:19:56

USER PROMPT:

AI ASSISTANT RESPONSE:

The **ResumeForge ATS** web application is built and running.

■ Live Application Access
The server is running locally and accessible at:
■ **http://127.0.0.1:8000**

*(To start the server at any time in the future, run **py run.py** from the project folder).*

■ Key Features Built

1. **Job Description Input (Text & Links)**:
- **Paste JD**: Direct text area with word counter and instant target skills detection tags.
- **Fetch from URL**: Built-in scraper using BeautifulSoup to extract job postings directly from career websites.
- **Quick 1-Click Samples**: Load sample roles (**"Full Stack SWE"** or **"Data Analyst"**) from the top bar to test instantly.

2. **Student CV / Profile Input**:
- **CV File Upload**: Drag-and-drop **.pdf**, **.docx**, or **.txt** resumes. The backend automatically extracts candidate info, skills, education, experience, and projects to pre-fill the form.
- **Interactive Manual Input**: Add/edit contact links, tags for skills, experience cards, project descriptions, and coursework.

3. **AI-Powered ATS Resume Tailoring**:
- Synthesizes a targeted professional summary matching the JD's role title.
- Categorizes skills into ATS-friendly groupings (*Languages*, *Frameworks*, *Cloud/Databases*, *Methodologies*).
- Enhances experience and project bullet points with strong action verbs and quantifiable metrics.

4. **Comprehensive ATS Scorecard & Live Optimization**:
- **Overall ATS Match Score** (0–100%) with animated circular gauge and rating badge.
- **Before vs. After Comparison**: Displays how much the score improved from the student's old CV to the tailored ATS resume.
- **Sub-score Breakdown**: Keyword Overlap %, Hard Skills Alignment %, Action Verbs & Metrics %, and ATS Formatting Health %.
- **Interactive Keyword Inspector**: Matched skills shown in green; missing skills shown in amber with a **1-click "+" button** to inject them straight into the resume.
- **In-Place Live Editing**: Toggle inline editing to tweak text directly on the resume sheet with real-time score recalculation.

5. **Multi-Format Export**:
- **Print / Save as PDF**: Clean ATS standard layout with zero UI chrome.
- **Download Word (.docx)**: Machine-readable **.docx** file using standard fonts and headings without tables.
- **Copy Plain Text**: Formatted for pasting into Taleo, Workday, or Greenhouse text boxes.

■ Source Files Created
- [**server.py**](file:///c:/Users/EdunetFoundation/Documents/Project/server.py): FastAPI backend and endpoints.
- [**ats_engine.py**](file:///c:/Users/EdunetFoundation/Documents/Project/ats_engine.py): ATS scoring algorithms, keyword analyzer, and DOCX generator.
- [**parsers.py**](file:///c:/Users/EdunetFoundation/Documents/Project/parsers.py): PDF/Word extraction, section parser, and URL scraper.
- [**run.py**](file:///c:/Users/EdunetFoundation/Documents/Project/run.py): Launcher script.
- [**static/index.html**](file:///c:/Users/EdunetFoundation/Documents/Project/static/index.html), [**static/styles.css**](file:///c:/Users/EdunetFoundation/Documents/Project/static/styles.css), [**static/app.js**](file:///c:/Users/EdunetFoundation/Documents/Project/static/app.js): Modern interactive UI.
- [**test_ats.py**](file:///c:/Users/EdunetFoundation/Documents/Project/test_ats.py) & [**test_api.py**](file:///c:/Users/EdunetFoundation/Documents/Project/test_api.py): Automated test suites (all passing).
- Detailed documentation: [**walkthrough.md**](file:///C:/Users/EdunetFoundation/.gemini/antigravity/brain/a1c97eaa-eed3-449d-bc27-5a68bc0989e3/walkthrough.md) and [**README.md**](file:///c:/Users/EdunetFoundation/Documents/Project/README.md).

Key Operations: *■ write_to_file | ■ run_command | ■ replace_file_content | ■ manage_task*

TURN #3 — 2026-09-04 10:33:08

USER PROMPT:
everything is good and workinng now i want you to integrate this into my git and publish the project

Key Operations: *■ run_command | ■ list_dir*

TURN #4 — 2026-09-04 10:33:37

USER PROMPT:
everything is good and workinng now i want you to integrate this into my github account and publish the project

AI ASSISTANT RESPONSE:
I have prepared the project and completed the local Git setup:

1. **Git & GitHub CLI installed**: Configured **git** (**v2.55.0**) and **gh** (**v2.100.0**) in your user environment.
2. **Repository Initialized**: Added **.gitignore**, **requirements.txt**, **README.md**, and all application files.
3. **Committed**: Created the initial commit on branch **main** (**43b9b02**).

To connect and publish to your GitHub account:

Please choose the method you prefer:

Method 1: Using a GitHub Personal Access Token (Fastest)
1. Generate a token on GitHub: **GitHub Settings $\rightarrow$ Developer Settings $\rightarrow$ Personal Access Tokens (Tokens classic)** $\rightarrow$ Check **repo** scope.
2. Reply here with your **GitHub Username**, **Repository Name**, and your **Token** (or paste the remote URL):
[Code:
Username: your-github-username
Repository Name: ResumeForge-ATS (or any name you prefer)
Token: ghp_xxxxxxxxxxxx

]
I will immediately create the remote, configure credentials, and push the repository for you!

Method 2: Using GitHub CLI Web Authentication
If you prefer to sign in via your browser, run this command in your PowerShell terminal:
[Code:
gh auth login
]
Select **GitHub.com** $\rightarrow$ **HTTPS** $\rightarrow$ **Login with a web browser**. It will give you a 1-time code and open your browser to authorize. Once done, let me know and I will run **gh repo create** and push it to your account immediately!

Method 3: If you already created an empty GitHub repository
If you already created a repository on github.com/new, just share the repository URL (e.g., **https://github.com/your-username/your-repo.git**) and I'll push to it.

Key Operations: *■ run_command | ■ manage_task | ■ write_to_file*

TURN #5 — 2026-09-04 10:51:53

USER PROMPT:
username: mupanpruthvi
Repo: ATS resume
token :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

AI ASSISTANT RESPONSE:
■ **Your project is officially integrated and published to your GitHub account!**

■ Live GitHub Repository:
■ **https://github.com/mupanpruthvi/ATS-Resume**

■ Summary of Actions Completed:
1. **GitHub Repository Creation**: Created public repository **ATS-Resume** under your account (**mupanpruthvi**).
2. **Author Attribution**: Set author to **mupanpruthvi <mupanpruthvi@users.noreply.github.com>**.
3. **Pushed Code**: Successfully pushed the complete codebase to branch **main** on GitHub:
- Full-stack FastAPI backend (**server.py**, **ats_engine.py**, **parsers.py**).
- Modern responsive single-page frontend (**static/index.html**, **static/styles.css**, **static/app.js**).
- Test suites (**test_ats.py**, **test_api.py**).
- Project documentation (**README.md**, **.gitignore**, **requirements.txt**).
4. **Secure Credential Setup**: Configured local Git credential store so any future **git push** or **git pull** from this machine will authenticate automatically.

■ Local App Status:
- Server is currently running at: **http://127.0.0.1:8000**
- To start it anytime in the future, run:
[Code:
py run.py
]

Key Operations: *■ run_command | ■ replace_file_content*

TURN #6 — 2026-09-04 10:54:20

USER PROMPT:
i have a doubt

AI ASSISTANT RESPONSE:
Please go ahead! What doubt or question do you have? I'm here to help.

TURN #7 — 2026-09-04 10:55:05

USER PROMPT:
if any other people oen this link http://127.0.0.1:8000 will they can do it

AI ASSISTANT RESPONSE:
No, other people cannot open http://127.0.0.1:8000.

Why?
- **127.0.0.1** (also known as **localhost**) is a special computer address that means **"this computer only"** (the loopback address).
- If your friend or student types **http://127.0.0.1:8000** into their phone or laptop, their device searches **inside their own machine**, not yours!

How to make it accessible to other people:

Here are the **3 ways** you can share it:

---

1. Share with anyone on the **Same Wi-Fi Network** (Instant & Free)
If other people or your phone are connected to the same Wi-Fi network as your laptop:

1. In your **server.py** or terminal, start the server listening on all network interfaces (**0.0.0.0**):
[Code:
py -m uvicorn server:app --host 0.0.0.0 --port 8000
]
2. Your laptop's Wi-Fi IP address is:
■ **http://192.168.1.9:8000**
3. Anyone connected to your Wi-Fi (or your phone) can type **http://192.168.1.9:8000** in their browser and use the website!

---

2. Share with anyone in the **World** using a free tunnel (e.g. **localtunnel** or **ngrok**)
If you want to share a quick link with a friend anywhere in the world right now while your computer is running:
- You can create a temporary public web link using tools like **Localtunnel** or **ngrok**:
[Code:
npx localtunnel --port 8000
]
It gives you a public link like **https://clean-cats-jump.loca.lt** that anyone can open anywhere.

---

3. Deploy it to the Cloud for **24/7 Free Worldwide Access** (Recommended! ■)
Since your project is already pushed to your GitHub:
■ **https://github.com/mupanpruthvi/ATS-Resume**

You can host it online permanently for free on **[Render.com](https://render.com)** so anyone can use it anytime (even when your laptop is turned off):

1. Go to render.com and sign in with your GitHub account (**mupanpruthvi**).
2. Click **New +** $\rightarrow$ **Web Service**.
3. Select your repository: **ATS-Resume**.
4. Configure these 3 fields:
- **Environment**: **Python 3**
- **Build Command**: **pip install -r requirements.txt**
- **Start Command**: **uvicorn server:app --host 0.0.0.0 --port $PORT**
5. Click **Create Web Service** (Select the Free plan).

Within 2 minutes, Render will give you a live public HTTPS address like:
■ **https://ats-resume-xxxx.onrender.com**

Anyone in the world can open that link on their computer or mobile phone!

Would you like help setting up a free cloud deployment on Render, or setting it up for your local Wi-Fi?

Key Operations: ■ *run_command*

TURN #8 — 2026-09-04 10:57:01

USER PROMPT:
do 3 one and update the deployment in github tto the deployed link

AI ASSISTANT RESPONSE:
I have configured the cloud deployment setup for you!

1. What Has Been Added to Your GitHub Repo:
- **render.yaml**: Automated Blueprint file configured to run Python FastAPI on the cloud.
- **Procfile**: Web process launcher for cloud servers.
- **1-Click Deploy Button**: Added directly to your GitHub [**README.md**](https://github.com/mupanpruthvi/ATS-Resume#readme).
- **GitHub Topics & Tags**: Tagged your repository with **#ats**, **#resume-generator**, **#fastapi**, **#job-search**.

---

2. Launch Your Live Cloud Website (Takes ~60 seconds):

Because Render requires an account login to bind the free hosting to your name, simply click this 1-click link:

■ **[Deploy ATS-Resume to Render (1-Click)](https://render.com/deploy?repo=https://github.com/mupanpruthvi/ATS-Resume)**

1. Sign in with your GitHub account (**mupanpruthvi**).
2. Click **"Apply"** (Render will automatically read your **render.yaml** settings).
3. Within 1–2 minutes, Render will build and launch your website and provide your live public URL (for example: **https://resume-forge-ats-xxxx.onrender.com**).

---

3. Updating the Live Link on GitHub:
As soon as Render assigns your URL, reply with your link here (e.g., **https://resume-forge-ats-xxxx.onrender.com**), and I will immediately:
- Update the **Official Website Link** in your GitHub repository's **About** sidebar.
- Update the live link badge in your **README.md**.

Key Operations: ■ *run_command* | ■ *write_to_file* | ■ *replace_file_content*

TURN #9 — 2026-09-04 11:01:08

USER PROMPT:
https://resume-forge-ats.onrender.com this is the link

AI ASSISTANT RESPONSE:
■ **Congratulations! Your project is deployed and live worldwide!**

■ Live Application (Share with Anyone in the World):
■ **https://resume-forge-ats.onrender.com**

Anyone on any device (phone, laptop, tablet) anywhere in the world can now open this link and:
- Paste job descriptions or job links from any company or career portal.
- Upload their old CV or type skills/experience.
- Generate ATS-tailored resumes in real time.
- View ATS match scores, missing keywords, and download ATS PDF / DOCX resumes!

■ GitHub Repository Updated:
■ **https://github.com/mupanpruthvi/ATS-Resume**

- **Official Website Link**: Set on your GitHub repository header & **About** section.
- **Live Demo Badge**: Added to the top of your **README.md**.
- **Status**: ■ **Online & Responding (HTTP 200)**.

Key Operations: ■ *run_command* | ■ *replace_file_content*

TURN #10 — 2026-09-04 11:06:42

USER PROMPT:
see what ever the converstion i have started for this project right from begining i need a document in pdf of athhe deatiled conersation

Key Operations: ■ *run_command* | ■ *write_to_file* | ■ *replace_file_content*
